



Introducción Node Js

Proyecto de Producción

Modelo Servidor - Cliente

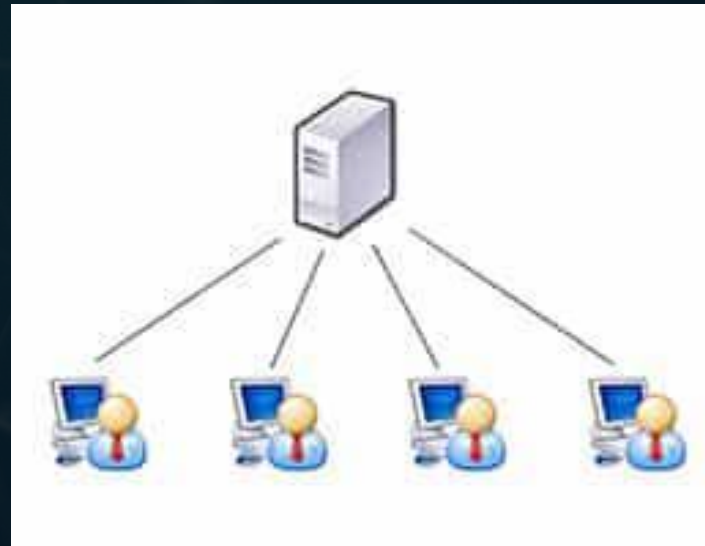
¿Quién es el cliente?

¿Quién es el servidor?

En este modelo, la comunicación siempre toma la forma de parejas de solicitud-respuesta, siempre iniciadas por el cliente y nunca por el servidor.

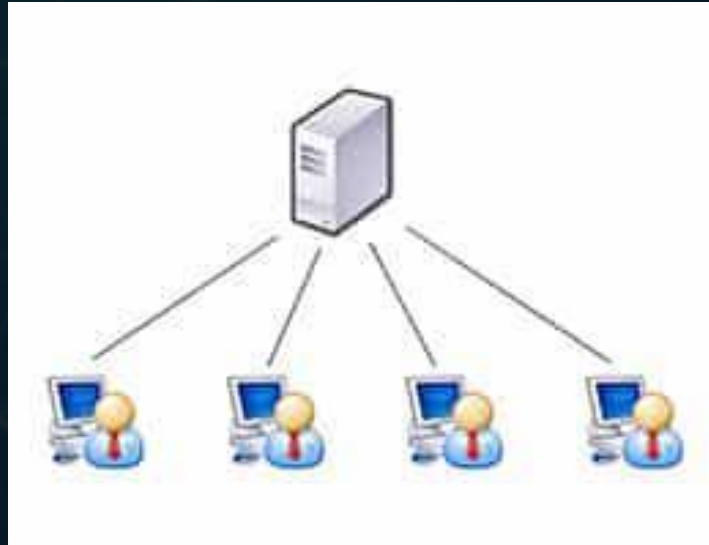
Los clientes son aquellos que crearán la consulta y le solicitarán algo al servidor (computadora, celulares).

Los servidores son aquellas computadoras dispuestas para responder solicitudes. Están todo el tiempo escuchando en busca de pedidos y al obtenerlos buscan lo solicitado y se lo envían al cliente.



Modelo Servidor - Cliente

En nuestro caso, tanto el cliente como el servidor van a estar corriendo en la misma máquina como 2 programas separados. Cada uno tendrá su código JavaScript propio y su consola de ejecución propia



Comunicación



CLIENTE



FRONTEND



Pedidos http:

- Get
- Post
- Put
- Delete

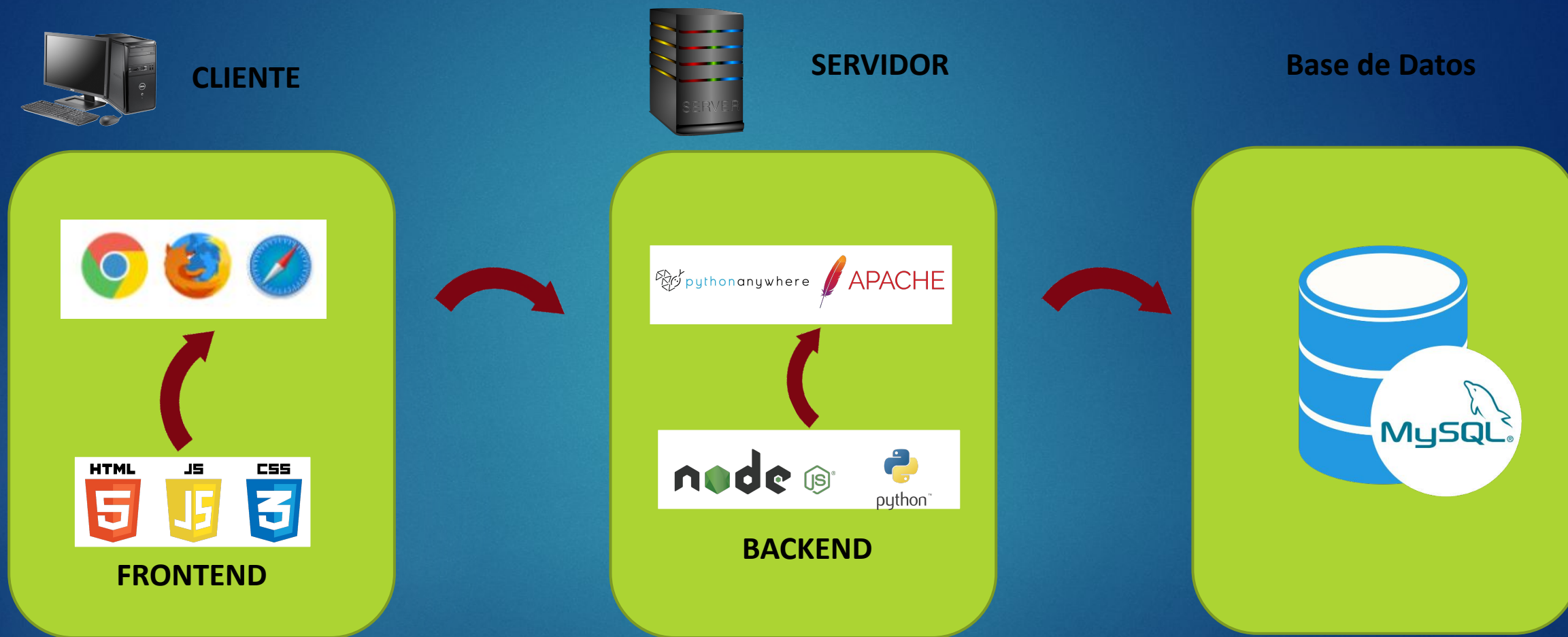


SERVIDOR



BACKEND

Comunicación - ¿Y qué hacemos con los datos?



Node.js como Backend

Node.js es un entorno JavaScript que nos **permite ejecutar en el servidor**, de manera asíncrona, con una arquitectura orientada a eventos y basado en el motor V8 de Google. Es una plataforma que avanza muy rápidamente y cada vez está más presente en el mercado.

Permite ejecutar en el servidor

Permite que usemos JS tanto en el cliente modificando el DOM, como en el servidor atendiendo y respondiendo pedidos. Antes era necesario usar diferentes lenguajes para cada cosa, ahora podemos usar JavaScript para todo lo referido a WEB tanto Frontend como Backend.

De manera asíncrona, con una arquitectura orientada a eventos

El servidor “espera” eventos, es decir, las peticiones de los clientes para luego responderlas.

El servidor no sabe cuando estas ocurrirán, entonces se dice que es asíncrono

¿Qué vamos a tener en el Backend?

Vamos a crear un tipo de aplicación denominada “**API REST**”, lo cual es un estándar hoy en día en cualquier software (páginas web, aplicaciones móviles, aplicaciones de escritorio, etc.) que deba intercambiar datos. **REST es cualquier interfaz entre sistemas que use el protocolo HTTP** (o HTTPS) para obtener datos o generar operaciones sobre esos datos. Nosotros lo vamos a usar para intercambiar datos con un servidor, pero puede servir también para intercambios entre aplicaciones dentro de una misma PC.

¿Qué características tiene una API REST?

Cada petición HTTP contiene toda la información necesaria para ejecutarla, lo que permite que ni cliente ni servidor necesiten recordar ningún estado previo para ejecutarla.

Los pedidos de una aplicación REST siempre se manipulan a partir de la URL. Esto hace que, para llegar a cierta información, sea único el identificador de cada recurso de ese sistema.

API REST siempre se adapta al tipo de sintaxis o plataformas con las que se esté trabajando. Con una API REST se pueden tener servidores NodeJS, PHP, Java, C# o Python. Lo único que es indispensable es que las respuestas a las peticiones se hagan siempre en el lenguaje de intercambio de información usado, normalmente XML o JSON.

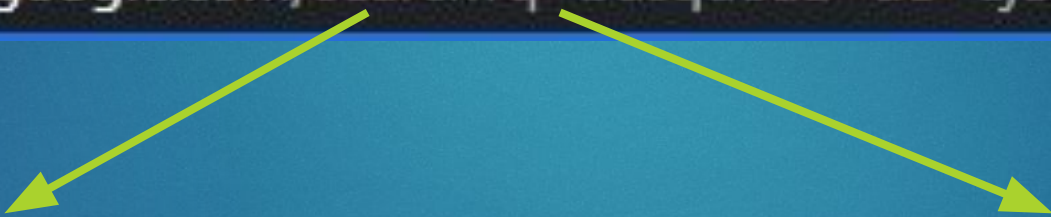
Pedido tipo GET

Son los pedidos HTTP básicos, toda la información viaja expuesta en el URL. Por lo cual, no son ideales en caso de pasar datos privados como pueden ser usuarios y contraseñas.

Por normativa, son los pedidos que se debería utilizar cuando se le quiere solicitar información al Backend.

Ejemplo de Pedido Get:

```
https://www.google.com/search?q=busqueda+de+ejemplo&rlz=1C1CHBF_es/
```



“/search”: Nombre del pedido que se atiende en el back (vamos a necesitar un app.get)

q: En este caso, es la variable que se pasa al Backend y “búsqueda de ejemplo” su valor

Pedido tipo POST

Son los pedidos HTTP básicos, toda la información viaja en un objeto, ya no está expuesta.

Por normativa, son los pedidos que se debería utilizar cuando se le quiere guardar información nueva en el Backend.

“/pedido”: El nombre del pedido es lo único que vamos a ver expuesto (necesitamos un app.post para atenderlo)

En este caso, no podemos ver las variables directamente en la URL

Iniciando un servidor

Lo primero que debemos verificar es si está instalado el node. Para lo cual, abrimos la consola de Windows y ejecutamos:

```
C:\Users\marti>node -v  
v14.17.4
```

Podemos verificar también la versión del npm:

```
C:\Users\marti>npm -v  
6.14.14
```

Iniciando un servidor

Ahora creamos una carpeta para nuestro servidor y desde la consola en ella ejecutamos el comando “npm init”.

Con esto nos van a saltar las siguientes opciones:

```
name: (nombre _de_la_app)
version: (1.0.0)
description: clase de introducción Node Js
entry point: (index.js)
test command: nodemon index.js (se va a ejecutar con el npm test)
git repository:
keywords: ES
author: martin
license: (ISC)
```

Luego instalamos por consola algunos módulos que nos son útiles:

```
npm install express --save
npm install body-parser --save
npm install nodemon --save
```


Archivo index.js

Nuestro archivo index.js es el archivo principal de nuestro servidor, el mismo va a empezar con la siguiente estructura:

```
var express = require('express');
var bodyParser = require('body-parser');

var app = express();
var port = process.env.PORT || 3000;

// Convierte una petición recibida (POST-GET...) a objeto JSON
app.use(bodyParser.urlencoded({extended:false}));
app.use(bodyParser.json());

app.get('/', function(req, res){
    res.status(200).send({
        message: 'GET Home route working fine!'
    });
});

app.listen(port, function(){
    console.log(`Server running in http://localhost:${port}`);
    console.log('Defined routes:');
    console.log('    [GET] http://localhost:3000/');
});
```

Usando SQL

Para comunicarnos con la base de datos, lo primero que necesitamos hacer es instalar el módulo de SQL:

```
npm install mysql2 -save
```

Y debemos importar el archivo mysql.js aportado por el docente:

```
const MySQL = require('./modulos/mysql.js')
```

Por último, vamos a poder hacer pedidos llamando a la función realizarQuery:

```
const respuesta = await MySQL.realizarQuery(`  
SELECT * FROM Grades;  
`)
```


Ejemplo Pedido Get en el Servidor

```
app.get('/obtenerEstudiantes', async function(req,res){
  //Los pedidos get reciben los datos del req.query
  console.log(req.query)
  const respuesta = await MySQL.realizarQuery(`
  SELECT * FROM Students;
  `)
  console.log({respuesta})
  //Le devuelve al cliente la respuesta de la base de datos
  res.send(respuesta)
})
```

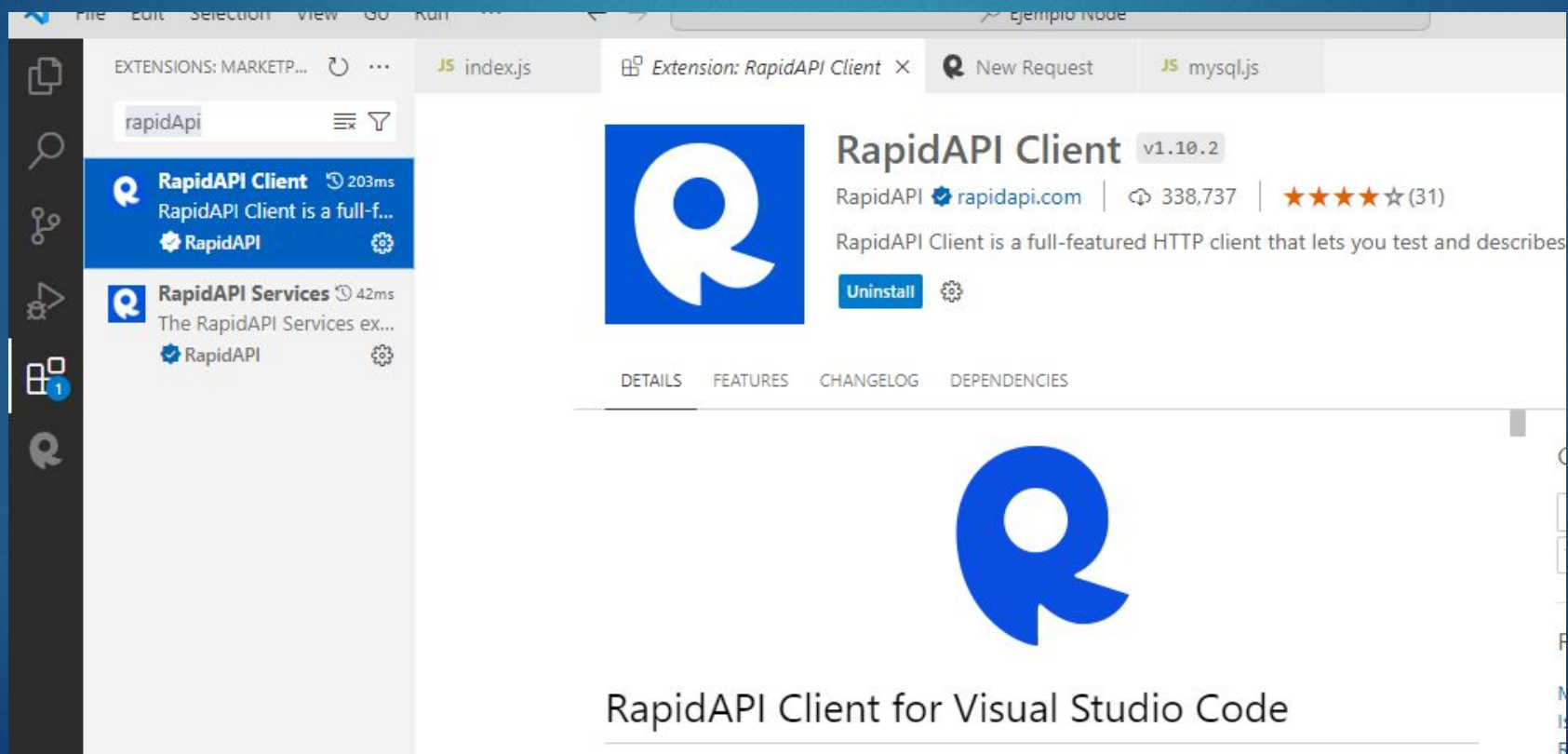
Ejemplo Pedido POST en el Servidor

```
app.post('/insertarEstudiante', async function(req,res) {  
  console.log(req.body)  
  //Los pedidos post reciben los datos del req.body  
  await MySQL.realizarQuery(`INSERT INTO Students (FirstName, LastName,  
    mail, id_grade)  
VALUES ('${req.body.FirstName}', '${req.body.LastName}',  
  '${req.body.mail}', ${req.body.id_grade})`);  
  res.send("ok") //Le respondo ok al cliente  
})
```


RapidApi Client

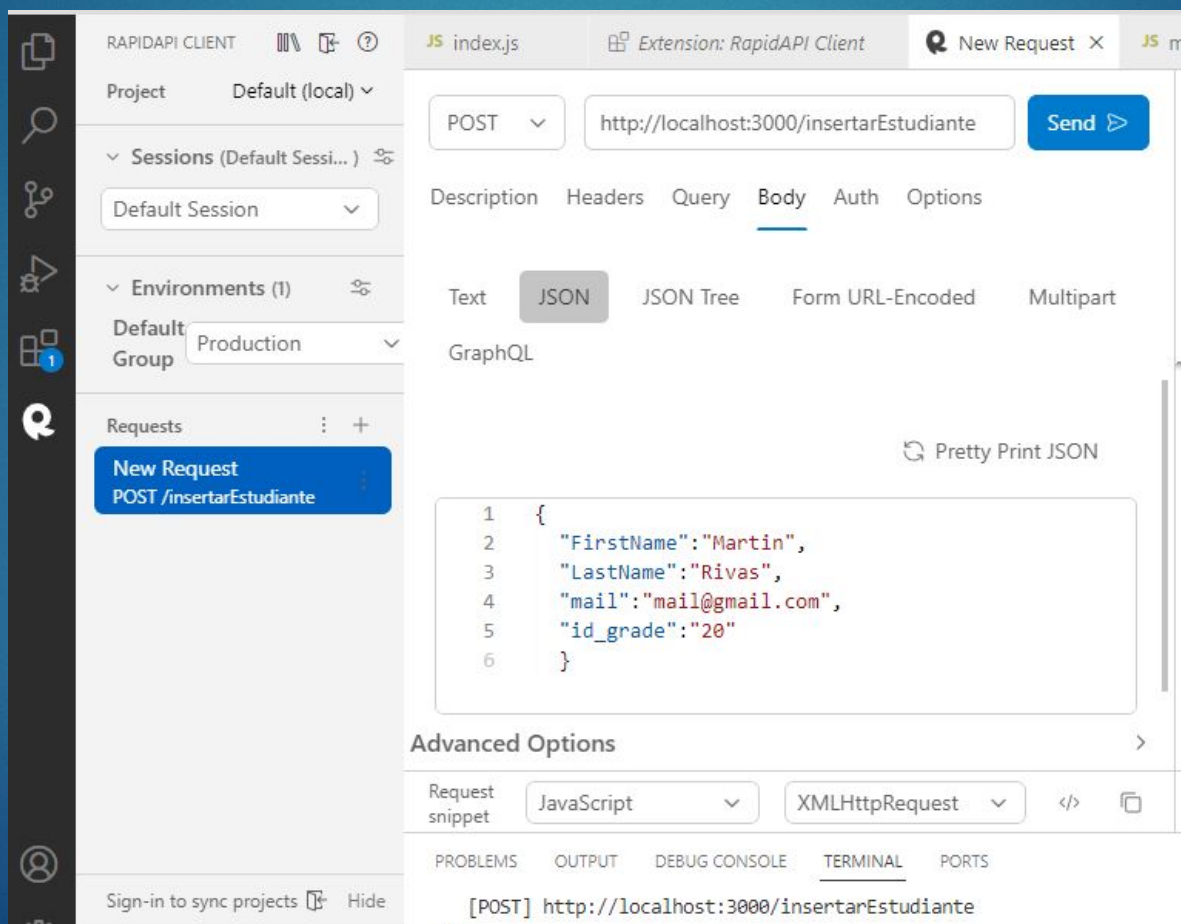
Para testear los pedidos POST, PUT y DELETE necesitamos instalar la extensión RapidApi para enviar estos tipos de pedidos.

Para instalarlo hay que ir a extensiones en el Visual y buscarlo



RapidApi Client

Y desde acá podemos mandar un pedido POST, enviando la información necesaria en el body



The background of the slide is a dark blue image of a multi-lane highway. Light trails from vehicles are visible, creating a sense of motion. A solid yellow rectangle is positioned in the top right corner.

Gracias